Very Fast 32 Bit Division With Single Word Divisors

Author: Alan Hunt, Date: 2026-10-10, Copyright © 2026

PICAXE Forum Post

I've created some very fast 32 bit division methods and thought I'd share.

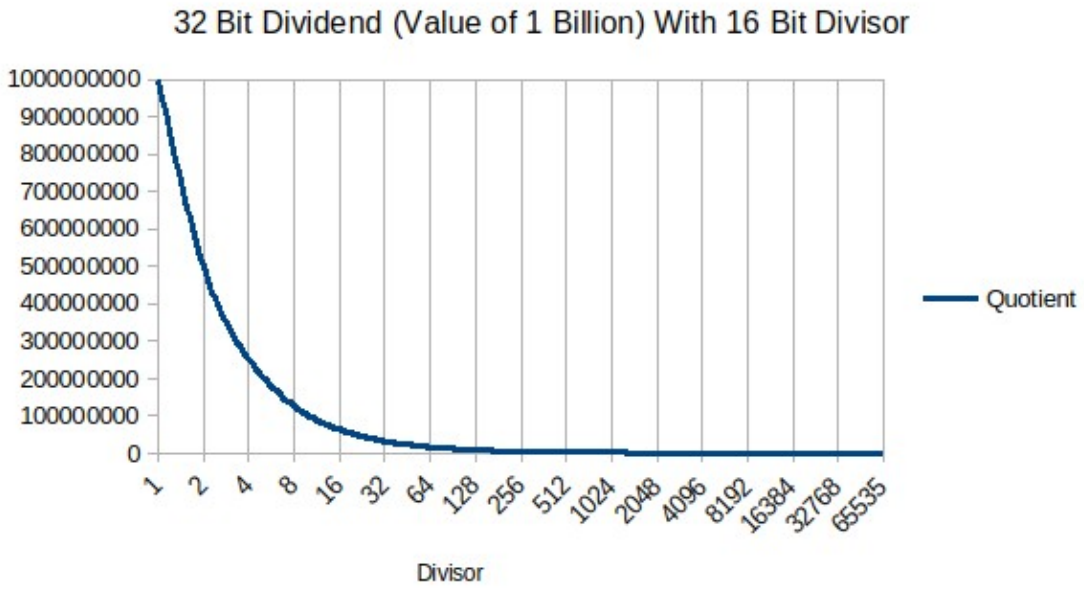
I've been working on a base ten logarithm function with decimal floating point numbers and to calculate from scratch was taking 300ms with a 14M2@4MHz. In an effort to speed things up I switched to an index warped lookup table, with more table entries at the bottom of a decade. The trouble was that this also involved division!!

Ultimately I achieved 32 bit division in only 21.4ms for truncating, or 23.4 ms for rounding. Back in 2012, Womai provided a solution that was refined by Allycat, which is based on simple division of the high word, followed by only 16 loops of single bit shifts and processing time was reduced to about 150ms. My two core solutions try to extend the division approach to the low word, in nibble sized chunks.

The "Nibbling Long Method" provides truncation and just does long division in 4 bit chunks. My "Nibbling Maths Method" provides rounding and does division on the low word, in addition to the high word; it then successively tries to eradicate the remainders by adding them to the low word while taking into account relative bit values and positions. It can't provide truncation in the normal sense because the low word remainder contains a fractional effect, even when you'd expect an integer result, but it is slightly more efficient in rounding to reduce quantization errors by half.

If you're waiting for the "initial" catch, here it is: The divisor is limited to the range of 0 to 4,096. The limitation prevents a division remainder being higher than 4,095, to allow multiplication by 16 for the next nibble. This worked for my needs because the maximum divisor was 2,695. It probably works for many other too, such the ADC values are only 10 bit.

I decided to pursue the Nibbling Maths Method for full 16 bit support and the graph below helps to explain very accurate results. When X reaches 256, the graph of 1/X is very flat and importantly it has virtually no curvature:



My solutions pre-scale large divisors down by a factor of 16 and post scale them for the final result. The “Nibbling Scalar Nearest Method” makes use of the curves flatness by efficiently choosing the nearest scaled divisor to provide results with better than 0.19% accuracy in 33.4 ms.

The “Nibbling Scalar Interpolation Method” makes use of the lack of curvature to calculate two divisions and linear interpolate them in only 70.5 ms with a maximum error of 0.000478%. As you can imagine interpolation results are almost always spot on! Both of these methods are also 100% accurate for divisors below 4097 and the time for this interpolation free route is 26.4 ms. The highest error is seen when the divisor is 4,104, half-way through the 1st scaled range.

The test program provides each nibbling method, along with test results an explanation. The core Nibbling Maths Method is shown below:

```
'Nibbling Maths Method (Rounding When Divisor < 4097)
'=====================================================

nibble = highWord // divisor * 16   'Modulus carries to lowWord.
highWord = highWord / divisor       'Divide high word in 1 step.
rounding = lowWord // divisor       'lowWord remainder for rounding.
lowWord = lowWord / divisor         'Divide low word in 1 step.
overflowCheck = lowWord             'For overflow checking lowWord.

'Process the top nibble of the lowWord.
lowWord.MSB = nibble / divisor * 16 + lowWord.MSB 'x16 aligns nibble.
nibble = nibble // divisor * 16         'x16 scales for next nibble.

'Repeat above but place the result in the next 4 bits.
lowWord.MSB = nibble / divisor + lowWord.MSB
nibble = nibble // divisor * 16         'x16 scales for next nibble.
```

```
'Repeat above but place the result in the next 4 bits.
lowWord = nibble / divisor * 16 + lowWord   'Word avoids overflow.
nibble = nibble // divisor * 16          'x16 scales for next nibble.

'Get the last 4 LSb from the highWord carry down.
lowWord = nibble / divisor + lowWord

'Roundup if necessary.
rounding = nibble // divisor + rounding * 2
if rounding >= divisor then : inc lowWord : endif

'Check for rare lowWord overflows.
if lowWord < overflowCheck then : inc highWord : endif
```

Theoretical Error Calculations:

Technique	Max Error	Notes
Divisor Truncated	+0.36%	Error ratio = (1/4096 – 1/4111) / (1/4096)
Divisor Rounded	+0.17% -0.19%	+Error ratio = (1/4096 – 1/4103) / (1/4096) -Error ratio = (1/4112 – 1/4104) / (1/4112)
Two Results Interpolated (n=8)	+0.00038%	Error ratio = $(4096^{-1} + (n / 16 * (4112^{-1} – 4096^{-1}))) / 4104^{-1}$

Measured Performance with a 14M2@4MHz:

Technique	Processing Time	Notes
Nibbling Long (Truncating)	21.4 ms	80 bytes. No error. Divisor limit 4096.
Nibbling Maths (Rounding)	**23.4 ms**	88 bytes. No error. Divisor limit 4096.
Nibbling Scalar Nearest	**26.3 ms** **33.4 ms**	133 bytes. No error for small divisors. (Divisors above 4095, max error 0.19%)
Nibbling Scalar Interpolate	26.4 ms 70.5 ms	280 bytes. No error for small divisors. (Divisors > 4095, max error 0.000478%)
Optimised 16 bit shifts	Approx 150 ms	80 bytes. Not measured or included here.
32 bit long division	Approx 300 ms	As above.

The division techniques are hungry in program space and for interpolation it's extreme. I made no attempt to reduce program space for the scalar methods, such as looping the process for the two interpolated results, or nesting the core nibbling method in a subroutine. Any nibbling technique can be defined in a macro to keep your program tidy, but program space will balloon if the macro appears more than once in your program. The alternative is a subroutine in which the code is stored once, but the `gosub` and `return` commands will add about 3.1ms to the figures above.

Master copies of software and documentation are held here.

<End of Document>